REALTIME SOURCE CODE ANALYZER

Project Overview

In this project, you will build a Source Code Analyzer, a powerful tool that acts as a senior developer paired with you to navigate complex codebases. Instead of spending hours reading through nested folders and undocumented functions, you will engineer a system where a user can simply provide a GitHub repository URL and instantly start asking natural-language questions about its architecture.

You will act as an AI Tools Engineer, building an application that automatically clones repositories, indexes Python source files, and embeds them into a vector store. You will tackle the unique challenge of chunking code—ensuring that classes and functions are kept intact for the AI to understand their logic.

Finally, you will wrap this Retrieval-Augmented Generation (RAG) engine in a Flask backend, creating a seamless web interface where developers can submit repos and engage in multi-turn, context-aware technical chats.

What you will Learn?

- **Code-Specific RAG:** Master building Retrieval-Augmented Generation pipelines optimized specifically for code understanding.
- **LangChain Operations:** Understand the complete workflow of document loading, chunking, and embedding generation using LangChain.
- **Vector Database Management:** Develop vector store operations using **ChromaDB** for the persistent indexing of large codebases.

- **Web App Development:** Implement a full web application using **Flask** to serve APIs and user interfaces.
- **Conversational AI:** Implement conversational memory chains to support fluid, multi-turn follow-up questions.

Why this Project?

- **High Utility:** Onboarding onto a new codebase is the most time-consuming part of a developer's job. You are building a tool that solves a massive, universal pain point in software engineering.
- **Advanced RAG Techniques:** Chunking code requires different strategies than chunking standard text (e.g., using Abstract Syntax Trees or code-specific splitters). This shows a deeper understanding of RAG architectures.
- **Full-Stack Competency:** By wrapping your AI logic in a Flask web application, you prove you can deliver accessible products, not just backend scripts.

Module
Foundations & Setup
Project Introduction and Overview
Project Repository & Setup
Project Architecture Discussions
Project Structure Creation
Local Package Setup
Configuration & Experimentation
How to get API keys
Project Notebook Experiment
Core Implementation
Implementation of Store index with ChromaDB
Implementation of chatbot with UI
Deployment
Dockerization and CICD Deployment

Tech Stack & Tools

1. **AI Framework:** LangChain
2. **Vector Database:** ChromaDB
3. **Backend & API:** Flask (Python)
4. **Data Source:** GitHub Repositories

5. **Embeddings & LLM:** OpenAI (or equivalent)